AOSP 进阶丛书 · 第十四册 · ANDROID 13 (API 33)

HAL 驱动级实战

AIDL HAL 五步构建模板（.aidl→Power.cpp→Android.bp→rc+VINTF+SELinux→客户端），三代终局图（legacy→HIDL→AIDL）、从 PowerManager 到内核的完整调用链、GKI 与 vendor_dlkm。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：从 API 到硬件驱动的最后一跳 2
第一章 HAL 三代终局 2
第二章 构建模板：五步走一个新 HAL 3
第三章 调用链：Framework 到 HAL 的完整跳 4
第四章 内核模块与设备树（BSP 视角） 4
第五章 设计决策七问 5
附录 A 验证五步 5
附录 B 快问快答（14 条） 5
附录 C 自检题 6
教科书习题集（分级） 6
术语表 7
排障手册：HAL 驱动级实战问题定位 7

本册速览

AIDL HAL 构建模板：cc_binary+init_rc+vintf_fragments+vendor:true
参考实现：hardware/interfaces/power/aidl/default/Power.cpp
VINTF 注册与 SELinux 三件套（file/service/sepolicy contexts）
调用链：Framework→ServiceManager→BpHw→HAL 进程
三代终局：legacy(同进程)→HIDL(隔离)→AIDL(大一统)

前言：从 API 到硬件驱动的最后一跳

HAL（Hardware Abstraction Layer）是系统与硬件的边界——书1 § 三讲了三代演进，本册教你**实际写一个**：定义接口→写参考实现→Android.bp→rc/VINTF/SELinux 三件套→客户端调用→端到端验证。以本树 `hardware/interfaces/power/aidl/default/` 的真实代码为模板。

第一章 HAL 三代终局



图 1 HAL 三代：legacy(同进程)→HIDL(隔离)→AIDL(大一统)，演进动机与读码区分法

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释HAL 驱动级实战的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 核心认知

"隔离"来自 vendor 分区+SELinux，不是来自 IDL 语言。HIDL 引入了进程隔离但代价是双工具链；AIDL 复用主线设施（/dev/binder+servicemanager）后工具链归一——**隔离没丢，只是"隔离的手段"从语言换成了分区+VINTF+策略。**

第二章 构建模板：五步走一个新 HAL



图 2 AIDL HAL 五步：定义 .aidl→参考实现→Android.bp→rc+VINTF+SELinux→客户端，以 power/aidl/default 为模板

2.1 Android.bp 逐行注释（真实源码）

text hardware/interfaces/power/aidl/default/Android.bp（节选+讲解注释）

```
aidl_interface {
name: "android.hardware.power", // 模块名：生成四后端（java/cpp/ndk/rust）
vendor_available: true, // 允许 vendor 分区引用
srcs: ["android/hardware/power/*.aidl"],
// 讲解注释①：接口版本冻结在 aidl_api/1, aidl_api/2, ...
// 新增方法必须开新版本目录（stable AIDL 的兼容保证）
}
cc_binary {
name: "android.hardware.power-service.example", // 产物名
relative_install_path: "hw", // → /vendor/bin/hw/
init_rc: [":android.hardware.power.rc"], // 讲解注释②：init 由此找到启动脚本
vintf_fragments: [":android.hardware.power.xml"], // 讲解注释③：VINTF 清单
vendor: true, // 归入 vendor 分区（书1 §七）
```

```
shared_libs: [
"libbinder_ndk", // AIDL C++ 的 NDK 库
"liblog",
],
srcs: ["Power.cpp", "main.cpp"],
}
```

2.2 三件套逐行

text rc + VINTF + SELinux（模板）

```
// mysvc.rc
service mysvc-hal /vendor/bin/hw/android.hardware.mysvc-service
class hal // init 的 hal 类启动（书2 §四 late-init 的 early_hal/hal 分支）
user system • group system
seclabel u:r:hal_mysvc:s0 // SELinux 域
// mysvc.xml（VINTF）
<hal format="aidl">
<name>android.hardware.mysvc</name>
<fqname>IMysvc/default</fqname>
</hal>
// SELinux 三件（system/sepolicy/vendor/）
mysvc.te: type hal_mysvc, domain;
file_contexts: /vendor/bin/hw/android.hardware.mysvc-service u:object_r:hal_mysvc_exec:s0
service_contexts: android.hardware.mysvc.IMysvc/default u:object_r:hal_mysvc_service:s0
```

第三章 调用链：Framework 到 HAL 的完整跳

text 全链拆解（以 PowerManager→Power HAL 为例）

```
Java: PowerManager.setPowerSaveMode(true)
→ IPowerManager.Stub.Proxy（AIDL 生成物）
→ BinderProxy.transact（书3 十跳前半）
→ 驱动 → vendor 进程的 binder 线程
→ BnPower::onTransact（AIDL 生成的 native Stub）
→ Power.cpp 的 setMode(bool) 实现
→ 写 sysfs /proc/cpuinfo/…（内核级生效）
```

设计洞察 • 逐段精读：为什么 HAL 在 vendor 进程

书1 Treble 的核心：vendor 代码（SoC 相关）不进 system 分区——升级 system 不必重编 vendor。HAL 进程的 SELinux 域（hal_xxx）与 framework 域隔离，授权通过 service_contexts+allow 规则显式放行。

第四章 内核模块与设备树（BSP 视角）

HAL 之下是内核世界：**设备树**（dtb/dtbo，硬件拓扑描述——boot.img 分区家族的一员）与**内核模块**（vendor_dlkm，.ko 文件由 insmod 加载）。读码视角：HAL 通过 sysfs/procfs 或 ioctl 与内核驱动对话（书1 § 二的 Android 内核增强）。本树的 `device/google/*-kernel/` 展示了预编译内核+模块的布局（Image.lz4 + dtbo.img + *.ko）。

第五章 设计决策七问

- **为什么新 HAL 一律 AIDL？** 工具链归一（一套编译器/调试/线程模型）；HIDL 仅维存量——工程上"少一套"压倒迁移成本。
- **为什么 HAL 接口要版本冻结（aidl_api/1,2,…）？** OTA 兼容性：老版本 framework 必须能调新版本 HAL——stable AIDL 由版本目录保证。
- **为什么 rc 里 class hal 而非 main？** 启动时序：HAL 必须在 framework 服务之前就绪（early_hal 先于 hal 先于 main）。
- **为什么 VINTF 是 XML 而非代码声明？** 兼容匹配需要机器可读：framework 的 compatibility_matrices 与 vendor 的 manifest 在构建期对齐（不匹配=拒绝启动）。
- **为什么 HAL 用 sysfs/procfs 而非直接 mmap 硬件寄存器？** 安全与稳定：内核驱动做权限仲裁+中断处理，HAL 只是"策略到硬件的翻译器"。
- **为什么 vendor_dlkm 独立于 boot？** GKI：通用内核不变，厂商差异全走模块+设备树——内核碎片化治理的镜像版 Treble。
- **为什么 passthrough 模式几乎不推荐？** 同进程=厂商崩溃连坐 framework——进程隔离是 Treble 的底线，性能收益不值得。

附录 A 验证五步

步	命令	通过标准	
①构建	m android.hardware.mysvc	BUILD SUCCESS	
②部署	adb sync 或刷 vendor.img	service list 有 mysvc	
③权限	adb shell ls -Z /vendor/bin/hw/mysvc*	seclabel 正确	
④调用	adb shell cmd mysvc_client --test	返回值正确	
⑤SELinux	logcat -b events |	grep avc	无 denied

附录 B 快问快答（14 条）

- 新 HAL 用什么 IDL？——AIDL（HIDL 冻结）。
- AIDL HAL 走哪个 binder 设备？——/dev/binder（主线域）。
- HIDL 走哪个？——/dev/hwbinder+hwservicemanager（遗留域）。

- AIDL 接口版本冻结在哪？——aidl_api/1,2,… 目录。
- HAL 的 Android.bp 关键四属性？——init_rc+vintf_fragments+vendor+relative_install_path。
- SELinux 三件？——.te+file_contexts+service_contexts。
- VINTF 的作用？——framework 与 vendor 接口版本匹配（不匹配不启动）。
- HAL 进程的 init 类？——class hal（或 early_hal 更早）。
- 为什么 HAL 在 vendor 分区？——升级 system 不必重编 vendor（Treble）。
- 内核模块放哪？——vendor_dlkm 分区（GKI 后）。
- 设备树是什么？——硬件拓扑描述（dtb/dtbo，boot 家族分区）。
- passthrough 是什么？——HIDL 的同进程旁路（几乎不推荐）。
- Sideband 是什么？——相机特权绕 SF 直叠硬件（书8 § 七）。
- 客户端怎么调 HAL？——ServiceManager.getService→Stub.asInterface→方法调用。

附录 C 自检题

- 默写五步走一个 AIDL HAL（含 rc/VINTF/SELinux 内容）。
- 从 PowerManager.setPowerSaveMode 到内核的每跳。
- 为什么 aidl_interface 的版本目录是兼容性保证?
- 读完本树 hardware/interfaces/power/aidl/default 的 Android.bp——每个属性的作用。

教科书习题集（分级）

第一级：简答题（检验概念理解）

（从附录 B 快问快答中挑 5 题，闭卷手写答案。）

第二级：分析题（检验机制理解）

- **追踪调用链**：画出本册核心机制的完整调用图。
- **失败推演**：核心机制某步失败，系统会怎样?
- **跨册串联**：本册机制与 Binder 十跳（书3）的交汇点。

第三级：设计题

- **架构演进**：重构本册核心机制，你会怎么改?
- **排障方案**：设计分层定位方案。

术语表

（术语表：从附录 B 提取本册专属术语，按拼音排序。）

排障手册：HAL 驱动级实战问题定位

1. dumpsys 定位状态 → 2. logcat 查看日志 → 3. 深入各章。

深度实战：HAL 驱动级实战的排障场景分析

场景：HAL 服务起不来

1. logcat -s ServicerManager → 检查是否注册
2. logcat -b events | grep avc → 检查 SELinux denied
3. lshal --list | grep <name> → VINTF 匹配
4. 检查 rc 文件的 seclabel 和 user/group
5. 修复：补全 SELinux 三件套 + VINTF XML